

# An Empirical Study on Bayesian Network-based Approach for Test Case Prioritization

Siavash Mirarab and Ladan Tahvildari

Software Technologies Applied Research (STAR) Group

University of Waterloo, Ontario, Canada N2L 3G1

{smirarab, ltahvild}@uwaterloo.ca

## Abstract

*A cost effective approach to regression testing is to prioritize test cases from a previous version of a software system for the current release. We have previously introduced a new approach for test case prioritization using Bayesian Networks (BN) which integrates different types of information to estimate the probability of each test case finding bugs [12]. In this paper, we enhance our BN-based approach in two ways. First, we introduce a feedback mechanism and a new change information gathering strategy. Second, a comprehensive empirical study is performed to evaluate the performance of the approach and to identify the effects of using different parameters included in the technique. The study is performed on five open source Java objects. The obtained results show relative advantage of using feedback mechanism for some objects in terms of early fault detection. They also provide insight into costs and benefits of the various parameters used in the approach.*

## 1 Introduction

Evolution of software systems and its associated challenges in testing motivates the testers to reuse test cases from one version to another. Regression testing techniques provide systematic ways to re-run all or some of the test cases from a previous version to validate the functionality of the current code. Test case selection [16], and test case prioritization [7, 17, 21] are well-known strategies to address this problem. Prioritization techniques order the test cases based on certain criteria such that the test cases with a higher chance of finding defects are executed earlier. These techniques help testers adjust their time and budget by running as many test cases as they can afford in a given period of time.

Researchers have introduced many techniques for prioritization [7, 9, 11, 18, 19, 20]. Most recently, we introduced a probabilistic approach to this problem which uti-

lizes Bayesian Networks to predict the probability of each test case finding a bug using different available sources of information [12]. Our initial experiments on this approach provided encouraging results. We compared one specific implementation of the approach to some other common techniques from the literature on one Java program and using a limited number of hand seeded faults. The reported experimental results served as a proof of concept, and hence, as mentioned in our earlier publication [12], further empirical studies are needed to better understand the characteristics of the proposed technique.

In this paper, we conduct a more extensive empirical study to evaluate the performance of the BN-based approach. The controlled experiments compare several different realizations of the proposed approach, on more objects, and using mutation technique which can provide large number of faults. The goal of our controlled experiments is to compare different implementations of the proposed approach and to understand the effects of its underlying parameters. The other important contribution of this paper is to incorporate a feedback mechanism to the proposed BN-based approach. Here, feedback means that after adding each test case to the ordered list, other test-cases which have common properties with it will have a lower probability to be selected. Most of the other prioritization techniques from the literature are reported to perform more efficiently when used in conjunction with feedback [7].

The rest of this article is organized as follows: In Section 2, a brief description of the test case prioritization problem is given. Section 3 presents the BN-based approach and describes how feedback mechanism can be used to enhance its efficiency. Section 4 introduces the research questions investigated in this study and then describes the design of the controlled experiment. Section 5 reports the obtained results. A discussion on the lessons learned from the study follows in Section 6. Finally, conclusions and future work wrap up the paper.

## 2 Test Case Prioritization Problem

Before describing our approach to the problem of test case prioritization, we present our formulation of the problem. Test case prioritization is traditionally defined as finding a permutation of test cases which maximizes an award function reflecting the goal of prioritization [17]. Similar to Kim *et al.* in [9], we take a probabilistic approach to this problem. Consider a set of random variables  $t_i$ , each of which reflect the outcome of a test case  $T_i$  from the test-suite  $\mathbb{T}$ . These random variables have two possible values: “Success” (meaning that a defect is detected) and “Failure”. The event of “Success” in  $T_i$  is denoted as  $t_i$ . The “prioritization technique” (in our approach) is a systematic way to estimate the probability of success for each test case  $T_i$  in  $\mathbb{T}$ . It builds a probability model which can be used by testers to order the test cases and run a subset of  $\mathbb{T}$  according to their time, budget, and other potential constraints that derive their testing process. A simple way to use the model is to order the test cases according to their probabilities of success and then execute them in that order until the time constraint forces the termination of regression testing.

Also, of particular interest is the notion of *feedback* in a prioritization technique. Feedback means updating the system model after adding each test case to the ordered list. Techniques using feedback rely on a greedy and iterative approach. After adding each test case, they estimate the effects of this selection and take into account these effects in deciding about subsequent selections. For example, assume we have a system with six elements:  $e_1 \dots e_6$  and the coverage relations between test cases and elements are as follows:  $t_1 \rightarrow \{e_2, e_5\}$ ,  $t_2 \rightarrow \{e_1, e_3\}$ ,  $t_3 \rightarrow \{e_4, e_5, e_6\}$ . Assume that the prioritization is merely based on how many elements a test case covers. In this scenario, the first chosen test case would be  $t_3$  because it covers three elements, while the others cover two elements each. After selecting  $t_3$ , two test cases are left, both of which cover two elements. In the absence of feedback, we would choose randomly between the remaining two. However, we know that  $e_5$  is already covered by  $t_3$ ; therefore  $t_1$  has merely one *additional* coverage, whereas  $t_2$  has two. After adding  $t_3$ , we can update our model of coverage information such that the already tested elements do not effect subsequent selections. This allows choosing  $t_2$  before  $t_1$  based on its *additional* coverage. This notion of using *additional* coverage is what feedback mechanism provides, and hence, techniques relying on feedback are often called *additional*.

## 3 BN Approach to Test Case Prioritization

Figure 1 illustrates the high-level process of our BN-based approach for test case prioritization. The first step, *Extracting Evidences*, gathers all the useful information that is to be included in the model. Our current solution exploits

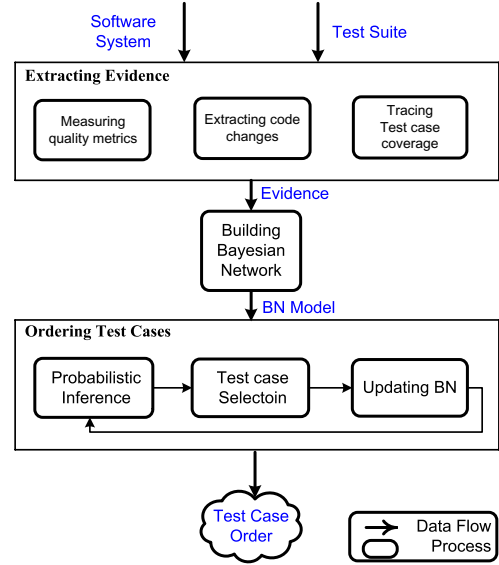


Figure 1. BN Approach: a Process Model.

three sources of information: *quality metrics*, *code change*, *test case coverage* (this could be expanded based on the problem domain). The next step is *Building Bayesian Network*, in which we build an inclusive probabilistic model to relate all the gathered information. The third step, *Ordering Test Cases*, consists of three tasks. First, *Probabilistic Inference* algorithms are used to associate to each test case its probability of success according to the constructed model. Then, *Test Case Selection* chooses some test cases and adds them to the ordered list. Those test cases will be then used in *Updating BN* to modify the model such that the probability of success of other test cases which cover similar parts of the code decreases. This new model will be fed back to the *Probabilistic Inference* step and the loop continues until all the test cases are ordered. This loop is where the feedback mechanism comes to the picture.

The first step (*Extracting Evidences*) is extensively studied by others, and we will make use of the existing contributions to implement it. In the rest of this section, we first elaborate on the second step (*Building the Bayesian*), including a brief introduction to Bayesian Networks, and then explain the third step (*Ordering Test Cases*) where the feedback mechanism is further clarified.

### 3.1 Building Bayesian Network

A Bayesian Network (BN) [14] is a directed acyclic graph consisting of three elements: *nodes* representing random variables, *arcs* representing probabilistic dependencies among those variables, and a *Conditional Probability Distribution Table (CPT)* for each variable. Some nodes can be evidence variables, those for which we know their value

(i.e. they are observed). Arcs specify the conditional probabilities between variables and can be thought of as indicators of causal relations. Each node has a table called CPT which includes the conditional probabilities of outcomes of its variable given the values of all its parents.

Bayesian Networks reflect the expert belief about the problem domain. They can be used to answer probabilistic queries. For example, based on the evidence (observed) variables, the posterior probability distributions of some other variables can be computed (probabilistic inference). There are two facets to building a BN: designing the structure and computing the parameters. Regarding the first issue, the notions of conditional independence and causal relation can be of great help. Intuitively, two events (variables) are conditionally independent if knowing the value of some other variables makes the outcomes of those two events independent. For computing the parameters, expert knowledge, statistical learning, and probabilistic estimations can be used. One potential problem is that we may know how a variable is dependent on each of its parents, but we do not have its distribution conditioned on all the parents. In such situations, the “noisy-OR” assumption can be helpful. This assumption gives the interaction between the parents and the child a causal interpretation and assumes that all causes (parents) are independent of each other in terms of their influence on the child [14].

Modeling is the main focus in solving the problems using BN. In Figure 2, the overall structure of the model is illustrated. Three elements of a BN (nodes, arcs, and CPTs) are designed in our approach as follows.

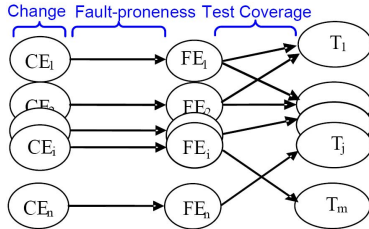


Figure 2. The Structure of the BN.

### 3.1.1 Nodes

As shown in Figure 2, our model has three types of nodes:

- **ce**: These set of variables represent changes in elements of the program. Each software element in the considered level of granularity (i.e. a class) has a node of this type. These variables can take a value of “Changed” (denoted by  $ce_i$ ) or “Unchanged” (denoted by  $\neg ce_i$ ).
- **fe**: This category reflects our belief whether each element is faulty. Each element of the program has one node of this type and each node can have the values of “Faulty” ( $fe_i$ ), or “Non-Faulty” ( $\neg fe_i$ ).

- **t**: These variables represent the outcome of a test case which can be “Success”( $t_i$ ) or “Failure”( $\neg t_i$ ). Each test case has one node of this type.

### 3.1.2 Arcs

Each arc in a BN indicates a causal relation between connected nodes. There are two sets of arcs in our network:

- **ce – fe**: Each fe node is the child of the corresponding (i.e. of the same code element) ce node. The existence of these arcs reflect the causal relation that changes to elements of software can introduce faults in the same element. Note that changing one element can potentially result in a defect in other places. In this work, we have simplified the model by ignoring such interactions. Future works can easily capture that aspect by allowing arcs from a ce to any fe node, depending on the results of change impact analysis.
- **fe – t**: Each t node can be the child of some fe nodes. These arcs imply the causal relation between presence of faults in a given software element and success of the test cases that examines that element.

### 3.1.3 Conditional Probability Table

Each node type has its own Conditional Probability Table:

- $P(ce_i)$ : ce nodes are not the child of any other node, so their distribution is not conditional. A  $ce_i$  variable captures the amount of change in  $e_i$ . Different methods for measuring change is a subject of our experiments presented in Section 4.
- $P(fe_i|ce_i)$ : In general, the probability of presence of fault in software is called fault-proneness. It is empirically shown that one can approximately predict the fault-proneness of code using software quality metrics (e.g. [1]). We use those fault-proneness models to estimate the values in this table. The metrics to use for the fault-proneness models will also be part of our controlled experiments.
- $P(t_i|fe_1 \dots fe_n)$ : t nodes can have more than one parent. The probability of fault detection for each test case can be estimated using its coverage of faulty elements. Having test coverage values, we can estimate  $P(t_i|fe_j)$  with the percentage of the code element  $e_j$  covered by test case  $T_i$ . This way the relation between a test case and one single element can be estimated. However, to build a CPT, we need the probability of success for a test case conditioned on all combinations of values of its parent fe variables. In order to estimate these values, we make the noisy-OR assumption explained earlier. In simple terms, the assumption is that the relation of a test case to an element is independent from its relation to any other element. It can be argued that the ability of a test case to detect a fault in a given element is not related to its ability to reveal faults in other elements, and hence the noisy-OR assumption follows. Using the noisy-OR assumption and having all  $P(t_i|fe_j)$  values, we can find

$P(t_i | fe_1 \dots fe_n)$ . A detailed description of how the noisy-OR assumption works can be found in [14].

### 3.2 Ordering Test Cases

Once the BN model is built, it is used to order test cases. This task is done in an iterative loop in which: i) using probabilistic inference, the probability distribution of  $t_i$  variables are computed, ii) some test case are selected and added to the ordered list, and iii) the model is updated based on the added test cases. In the following, we further explain these sub-processes as illustrated in Figure 1.

**Probabilistic Inference:** In this step, we estimate the  $P(t_i)$  values using the probabilistic inference. A BN can be used to answer probabilistic queries. That is, given the value of some observed variables (evidence variables), we can compute the updated conditional probability distributions for the other variables. This process is called probabilistic inference which is a well studied subject and has many known algorithms [14]. The specific algorithm to be used will be a subject of our experiments. Here, we use probabilistic inference to deduce the distribution of  $t$  variables given the value of  $ce$  variables.

**Test Case Selection:** In this step, based on the probabilities from the previous step, a number of test cases are selected and added to the ordered list. A parameter called *stp* controls the number of added test cases. In one extreme, we can set the value of this parameter to be equal to the size of the entire test suit. In this case, there would be no feedback mechanism. In the other extreme, we can set this parameter to be equal to 1 and feedback is used in its full capacity. Any number in this range can be used to have the feedback mechanism in place, while performing the computation intensive task of inference less often. This parameter provides the flexibility to adjust the running time of the technique. If we need the prioritization task to finish fast, we will set the parameter to a high value; otherwise, lower values will be used. The value of this parameter will be one of the dimensions of the experiments in the next two sections.

**Updating Bayesian Network:** In this step, the variables corresponding to the test cases just added to the ordered list would be marked as “Failure” evidence nodes. As a result, the probability of being faulty for the elements covered by those test cases will be reduced. Consequently, the other test cases that test the same elements will have a decrease in the probability of success. Note that regardless of whether a test case succeeds or not, we mark its variable as a “Failure”. There are two reasons for this. First, during prioritization we do not know yet whether the test case is going to succeed. Second, if we mark a node as “Success”, then the other test cases that cover the same elements will have an increase in their probability of success. This is not desirable because we have already covered those elements at

least partially. The assumption is that in the event of a “Success”, the test process will be terminated to fix the bug and it is resumed only when this particular test case would pass.

## 4 Experiment Setup

This section presents how our empirical experiments are designed and performed. The followings are research questions that we are aiming to answer through these experiments:

- **RQ1:** What is the effect of adding feedback mechanism on the performance of the approach?
- **RQ2:** What is the effect of using different information gathering techniques (i.e. change information and fault-proneness models)?
- **RQ3:** What is the effect of adjusting parameters in the system (i.e. inference algorithm and *stp* parameter)?

To address these questions, a controlled experiment is designed and performed. In the rest of this section, we describe the experiment objects, variables and measures, experiment design, and finally, threats to validity.

### 4.1 Experiment Objects

We used five open source Java programs for this experiment: *ant*, *xml-security*, *jmeter*, *nanoxml*, and *galileo*. The first three objects have JUnit test suites, and the last two have TSL (Test Specification Language) suites [13]. These programs, and their test suites are all obtained from an infrastructure supporting experimentation [4]. This repository has been extensively used for controlled experimentations.

Table 4.1 lists the following attributes of our objects: “Versions” (the number of versions of the object program), “Classes” (the number of class files in the most recent version of the program), “Size (KLOCs)” (the total lines of code in the most recent version of the program), and “Test Cases” (the number of test cases available for the most recent version of the program).

To evaluate the performance of the proposed techniques, we needed object programs containing faults. In [6], the authors have added mutation faults to the mentioned object programs. We obtained the mutation faults from the authors of [6] and have used them for our experiments. The number

**Table 1.** Experiment Objects and Associated Data.

| Objects        | Versions | Classes | Size KLOC | Test Cases | Mutants |
|----------------|----------|---------|-----------|------------|---------|
| <i>ant</i>     | 9        | 627     | 80.4      | 877        | 412     |
| <i>jmeter</i>  | 6        | 389     | 43.4      | 78         | 386     |
| <i>xml</i>     | 4        | 143     | 16.3      | 83         | 246     |
| <i>nano</i>    | 6        | 26      | 7.6       | 216        | 204     |
| <i>galileo</i> | 16       | 87      | 15.2      | 912        | 2494    |



of mutation faults available for each of our object programs is shown in the last column of Table 4.1. It is noteworthy that as the focus is on the detection of regression faults (faults created by code modifications), all the mutations in each specific version fall within areas of code modified from the previous version to the current version.

## 4.2 Variables and Measures

### 4.2.1 Independent Variables

To investigate the aforementioned research questions, we need to define the following independent variables: “Feedback Mechanism”, “Change Information Gathering Technique”, “Fault-proneness Model”, “Inference Algorithm”, and “*stp* Parameter”.

**V1: Feedback Mechanism-** Decision on whether to use feedback mechanism produces two different techniques: “BN” refers to the case where feedback mechanism is not used, and “BNA” is the *additional* version of the approach where feedback is included.

**V2: Change Information Gathering Technique-** We evaluate two different alternative techniques for gathering change information: *Unix diff* and *Constant Pool diff*. *Unix diff* command can be used to match two consecutive versions of a source file against each other line by line to gather the change information in the form of the number of changed lines divided by the total number of lines. The other approach is to compare constant pools from the byte code; an approach which is originally designed to detect the similarity of two .class files. We used a tool called *Sandmark* which implements this technique [3]. The V2 variable capture these two techniques with the possible values of D (DIFF) and CP (Constant Pool).

**V3: Fault-proneness Model-** The fault-proneness models from literature (e.g. those surveyed in [1, 8]) can be divided into multivariate and univariate. Univariate models estimate fault-proneness using one single metric. Multivariate models, on the other hand, are a linear (or non-linear) combination of several univariate models. To use multivariate models, one should “train” the model on a second program (or earlier versions of the same program) and apply the potentially biased model to the system in question. As empirically evaluated in [1], using this approach, multivariate models do not necessarily generate better results; thus, in this work we use univariate models. In this experiment, we evaluate five common metrics from Chidamber and Kemerer suite [2]: CBO (Coupling between Objects), DIT (Depth in Tree), LCOM (Lack of Cohesion), RFC (Response for Class), and WMC (Weighted Method Complexity). Therefore, V3 variable is characterized by the selected metric suite which can have five different values.

**V4: Inference Algorithm-** Inference algorithms for a BN fall into two categories: *exact* (EXC) and *sampling*

(SMP) algorithms. We include one sampling algorithm (Epis [22]) and one exact algorithm (Lauritzen [10]) in this experiment.

**V5: *stp* Parameter-** The *stp* parameter introduced in Section 3.2 is also a subject of the experiment. We choose the following five values as the domain of this variable: 1, 5, 10, *adjustive* 10, and *adjustive* 20. The last two variables are *adjustive*, meaning that the parameter is set to 1 initially and then as the test cases are added to the ordered list, the value is increased gradually such that for the last test cases it reaches the specified number. For example, if we have 500 test cases with *adjustive* 10, *stp* is set to 1 initially. After 50 iterations (adding 50 test cases), this parameter is increased to 2. Then, after 25 more iterations when another 50 test cases are added, the parameter is increased to 3 and so on until the last 50 test cases where *stp* is set to 10. The advantage of this schema is that the earlier test cases in the ordered list are chosen more carefully.

### 4.2.2 Dependent Variable and Measures

To evaluate the effect of the independent variables, we use two main dependent variables: APFD (Average Percentage Faults Detected) [17] and running time of the algorithms. While the APFD metric provides insight to the benefit of each technique, the running time variable helps comparing the costs.

**APFD:** This metric calculates fault detection rate by measuring the weighted average of the percentage of faults detected over the test suite execution period. APFD values range from 0 to 100 where higher values indicate faster fault detection rates (the precise definition of APFD can be found in [17]).

**Running-time:** The running time of each implementation is computed on a windows system with a Pentium(R) 4 CPU and 2GB of RAM. The implementation is almost entirely in Java.

Although there are comprehensive models of cost-benefit analysis in the literature (e.g. [5]), we choose to use the above variables mainly because of two reasons. First, comprehending the meaning of these two variables is easier than the resulting numbers from those models. Second, due to the large similarity between the steps of all the alternative techniques under experiment, many of the factors in the cost-benefit models are identical among the alternatives. As a result, even if we use those models, the relative cost-benefit is mainly influenced by the running time and the APFD metrics.

## 4.3 Experiment Design

We have performed the controlled experiment in four different stages, each of which targets one of the variables V2 to V5. The feedback mechanism (V1) has been predicted

**Table 2.** Variable Values for the Four Stages.

| Var. | Stage 1                                  | Stage 2       | Stage 3                                                                    | Stage 4                                  |
|------|------------------------------------------|---------------|----------------------------------------------------------------------------|------------------------------------------|
| V1   | Target                                   | Target        | BNA                                                                        | BNA                                      |
| V2   | Target                                   | Constant Pool |                                                                            |                                          |
| V3   | CBO                                      | Target        | <i>ant, galileo</i> : DIT<br><i>xml</i> : RFC<br><i>jmeter, nano</i> : WMC |                                          |
| V4   | <i>ant</i> : Sampling<br>the rest: Exact |               | Target                                                                     | <i>ant</i> : Sampling<br>the rest: Exact |
| V5   | 1                                        |               |                                                                            | Target                                   |

to have a major impact, and hence, it has been treated differently. Both of V1 values (BN and BNA) are included in the first two stages. Besides V1, in each stage we fix the value of all the variables except the one targeted in that specific stage. For all the values of the variable under experiment, we compare the dependent variables. In some stages, *running-time* dependent variable is almost identical among all alternatives and we do not report that measure due to space limitations.

Table 2 lists for each variable, the fixed values in all the stages. The stage which concerns a specific variable is marked as “Target”. The choice of the actual value for a fixed variable in each stage is based on either our expectation of the optimal value, or based on the results from a previous stage. For example, CBO was chosen for V3 because it is widely reported to be a better indicator of fault-proneness (e.g. [8]). However, after finding the optimal values in Stage 2 for each specific object, we have used those values for Stages 3 and 4.

Our first research question (RQ1) is pursued through Stages 1 and 2 of the experiment where the feedback variable is included. The second research question is divided into two parts of “change information” and “fault-proneness model” which are addressed in Stages 1 and 2, respectively. The third research question is also divided into two parts: “inference algorithm” and “*stp* parameter” which are addressed in Stages 3 and 4, respectively. These two parameters are meaningful only for BNA<sup>1</sup>, and hence, V1 is set to BNA for the last two stages.

#### 4.4 Threats to Validity

In this section, we describe the construct, internal, and external threats to the validity of our study as well as the approaches we have used to limit the effects of these threats.

**Construct Validity.** The dependent measures used in this experiment are not complete in terms of capturing all the costs and benefits associated with the different alternatives. Performing accurate cost-benefit analysis is dependent on

<sup>1</sup>Because *stp* is meaningful only for BNA and the inference algorithm mostly affects running-time which is not an important issue for BN.

the specific context where a given technique is used. However, we have tried to capture both cost and benefit facets in the two dependent variables described earlier.

**Internal Validity.** There could be several other factors which influence the results and affect our inferences. The tools we have developed/used to collect data can potentially have defects. By validating our tools on several simple Java programs, we have aimed to restrict this threat. More importantly, the staged procedure of the experiment can influence the result. As the combination of all the values for different variables takes a large space, when we evaluate each specific variable, the value of the other variables are kept constant. Changing these values can (although unlikely) affect the results for the other variables. To restrict this problem, we have tried to choose the variables such that they have a small logical dependency on each other. Also, we have used the optimal values from one stage to the next. The reason is that the best combination of all values is likely to be similar to the combination of all the best values.

**External Validity.** There are also threats to the external validity of the experiment. The programs that we study are relatively small (7K-80K) and they are all open source software. Complex industrial programs with different characteristics may be subject to different concerns. Also, the testing process used for the open source software is not representative of all possible processes used in industry. The other threat to generalizing the findings is that we use mutation faults for evaluation purposes. Although using real faults is preferable, they are typically not available in large numbers; thus, researchers use faults created by mutation tools instead. Moreover, it is shown that mutation faults can be representative of real faults [6]. Finally, the tools we use in this study are prototypes and thus may not reflect tools used in a typical industrial environment. Methods to control these threats can be the subject of future studies with more programs (preferably from different domains), real faults, and better tools.

## 5 Obtained Results

In this section, we present the results from our experiment. Figure 3 plots an overview of the results for all the stages in the experiment. Each column represents one stage of the experiment and includes 5 boxplot diagrams corresponding to the five experiment objects. In each diagram, the independent variables corresponding to that stage are enumerated in the horizontal axis. The vertical axis is the APDF metric. For each version of an object, we randomly choose 15 sets of mutants each having a random size of 1 to 10. The boxplots illustrate the distribution of these 15 sets for all the versions of each object. In the reminder of this section, we look into the specifics of the results for each stage.

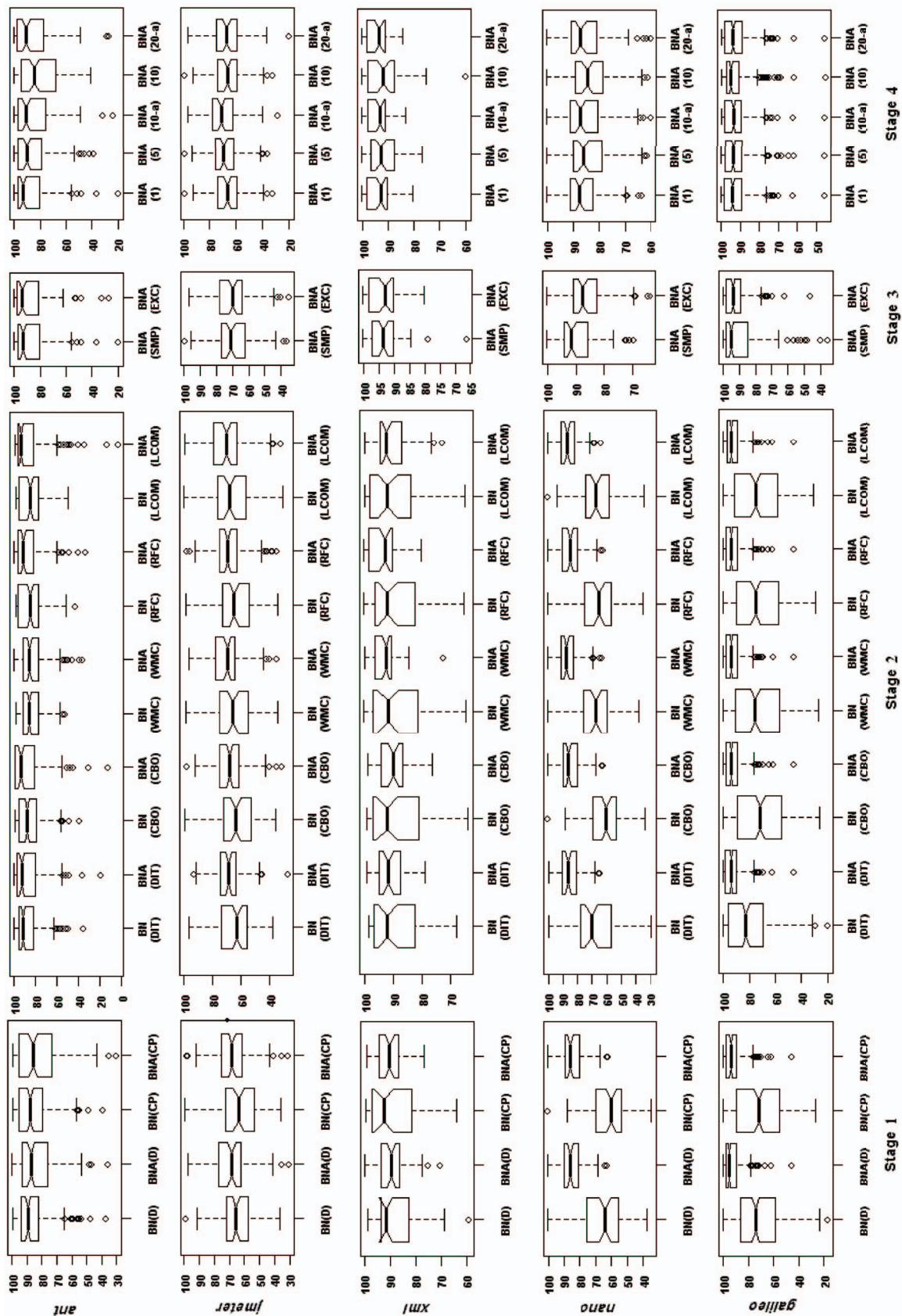


Figure 3. Boxplot Diagram of the APFD Values for All the 4 Stages



## 5.1 Stage 1

In this stage, we examine the effect of using different change information gathering techniques (V2) and also look into the effects of feedback mechanism (V1). The first column of the Figure 3 shows the results related to this stage. The letters in bracket indicate the values of V2 where D stands for Unix Diff technique and CP refers to the Constant Pool. Looking at the boxplots, V2 does not seem to affect the results but V1 has an important role. To validate the results, we use Kruskal-Wallis statistical test [15].

Table 3 illustrates the results of the test in two parts, one for V1 and one for V2. There is no single case of significant difference for V2, and hence, it is clear that this variable is not an important factor. Regarding V1, however, there are significant results. BNA results in better values of APFD for *nano* and *galileo*, the two objects with the TSL test suite. However, for the rest of objects (the ones with Junit test cases), the differences are not statistically significant.

**Table 3.** Kruskal-Wallis Test Results for Stage 1.

| V1             |                 |          |                   |          |
|----------------|-----------------|----------|-------------------|----------|
| Program        | BN(D) vs BNA(D) |          | BN(CP) vs BNA(CP) |          |
|                | chi.            | p-val.   | chi.              | p-val.   |
| <i>ant</i>     | 0.9             | 0.323    | 0.08              | 0.762    |
| <i>jmeter</i>  | 2.5             | 0.114    | 3.8               | 0.049    |
| <i>xml</i>     | 0.03            | 0.8      | 0.14              | 0.701    |
| <i>nanoxml</i> | 56              | < 0.0001 | 66                | < 0.0001 |
| <i>galileo</i> | 130             | < 0.0001 | 132               | < 0.0001 |

| V2             |                 |        |                   |        |
|----------------|-----------------|--------|-------------------|--------|
| Program        | BN(D) vs BN(CP) |        | BNA(D) vs BNA(CP) |        |
|                | chi.            | p-val. | chi.              | p-val. |
| <i>ant</i>     | 0.04            | 0.79   | 1.1               | 0.287  |
| <i>jmeter</i>  | 0.7             | 0.395  | 0.05              | 0.827  |
| <i>xml</i>     | 0.82            | 0.363  | 0.17              | 0.680  |
| <i>nanoxml</i> | 3.3             | 0.069  | 0.01              | 0.942  |
| <i>galileo</i> | 1.34            | 0.247  | 0.20              | 0.653  |

Table 4 reports the running-time of the four techniques. It is evident (as expected) that V2 does not have meaningful affect on this dependent variable. V1, on the other hand, has a significant impact on the running-time. The running-time of BNA technique is between 3 to 100 times larger than that of BN. Using feedback can assist early fault detection in some cases, but it does require longer running-time.

**Table 4.** Average Running-time (in sec), Stage 1.

| Program        | BN(D) | BNA(D) | BN(CP) | BNA(CP) |
|----------------|-------|--------|--------|---------|
| <i>ant</i>     | 5     | 462    | 5      | 497     |
| <i>jmeter</i>  | 6     | 71     | 4      | 66      |
| <i>xml</i>     | 0.7   | 2.2    | 0.7    | 2.2     |
| <i>nanoxml</i> | 2     | 6      | 2      | 6       |
| <i>galileo</i> | 6     | 71     | 5      | 66      |

## 5.2 Stage 2

The main concern in Stage 2 is the fault-proneness metrics (V3). The second column of Figure 3 illustrates the

results related to this stage for both BN and BNA techniques. The boxplot diagrams suggest that the V3 variable has only occasionally a significant impact. We use a five way Kruskal-Wallis statistical test (dt=4) to find out if the differences between the techniques are statistically significant. The results are shown in Table 5. Unlike Table 3, here we do not report results for V1 because the same pattern as that of Stage 1 is observed. To control for V3, we perform the test among the five BN and BNA alternatives separately. The results indicate that V3 dose not change the performance significantly in most of the cases.

**Table 5.** Kruskal-Wallis Test Results for Stage 2.

| Program        | BN   |          | BNA   |          |
|----------------|------|----------|-------|----------|
|                | chi. | p-val.   | chi.  | p-val.   |
| <i>ant</i>     | 7.6  | 0.109    | 30.44 | < 0.0001 |
| <i>jmeter</i>  | 4.3  | 0.36     | 2.4   | 0.65     |
| <i>xml</i>     | 0.06 | 0.963    | 13    | 0.011    |
| <i>nanoxml</i> | 12.4 | 0.015    | 2.6   | 0.623    |
| <i>galileo</i> | 30.7 | < 0.0001 | 1.3   | 0.861    |

In the two cases where significant differences are observed, the test is among five techniques. The interpretation should be that there is at least one technique which is statistically different from at least one other technique. To gain a better understanding, we performed Bonferroni adjustment [15] for the two cases of BNA in *ant* and BN in *galileo*. According to this test and in the significance level of 1%, for *ant*, WMC is significantly better than all the others but the rest are not different from each other. For *galileo*, DIT is significantly better than all the other alternatives but the rest are statistically similar.

## 5.3 Stage 3

In this stage, we are concerned with the inference algorithm (V4). The third column in Figure 3 shows the APFD values for this stage. The technique labelled as EXC uses the exact algorithm and the one labelled SMP uses a sampling algorithm. In principal, the APFD values achieved using different algorithms should not be much different. The boxplot diagram suggests that the actual results support this point except for *nano* object. Performing Kruskal-Wallis test for this specific object, we observed that the  $p - value$  is 0.007 which means the result is significant in 1% level but not in 0.1% level. Interestingly, here, the sampling algorithm has a better APFD value; an observation for which we have no solid justification.

**Table 6.** Average Running-time (in sec), Stage 3.

| Program  | <i>ant</i> | <i>jmeter</i> | <i>xml</i> | <i>nanoxml</i> | <i>galileo</i> |
|----------|------------|---------------|------------|----------------|----------------|
| BNA(EXC) | 871        | 2             | 1          | 8              | 66             |
| BNA(SMP) | 496        | 11            | 12         | 105            | 1434           |

The choice of inference algorithm influences the running-time to a large extent. Table 6 lists the average



running-time of the two BNA techniques. There is a mixed pattern observed in the results. While the exact algorithm (EXC) performs quite slower for *ant*, it is much faster for the other objects. This result is expected noting that as the size of the network grows, the sampling algorithm's complexity does not increase dramatically whereas in the exact algorithm, it can grow exponentially. Therefore, for small systems, the sampling algorithm is relatively slow (the time necessary for generating samples etc. is more than traversing the network), however for large systems, it is slower. Because *ant* is a notably larger system than the other objects, SMP performs faster on it. It is noteworthy that the specific exact algorithm used in this experiment traverses the network in a forward search fashion, and hence, the number of nodes in the first layer (i.e. number of classes) have a larger impact on the running-time than the nodes in the last layer (i.e. number of test cases).

#### 5.4 Stage 4

In this stage, we experiment with the *stp* parameter. The last column of Figure 3 depicts the boxplot diagrams related to this stage. The effect of *stp* parameter is expected to be two-fold. It can potentially affect APFD values and it certainly changes the running-time.

**Table 7.** Kruskal-Wallis Test Results for Stage 4.

|        | <i>ant</i> | <i>jmeter</i> | <i>xml</i> | <i>nanoxml</i> | <i>galileo</i> |
|--------|------------|---------------|------------|----------------|----------------|
| chi.   | 21.6       | 3.3           | 4.1        | 4.2            | 1.3            |
| p-val. | 0.0002     | 0.5           | 0.395      | 0.372          | 0.866          |

**Table 8.** Average Running-time (in sec), Stage 4.

|      | <i>ant</i> | <i>jmeter</i> | <i>xml</i> | <i>nanoxml</i> | <i>galileo</i> |
|------|------------|---------------|------------|----------------|----------------|
| 1    | 496        | 3.3           | 1.2        | 6              | 66             |
| 5    | 98         | 0.5           | 0.8        | 3              | 16             |
| 10-a | 138        | 0.5           | 0.9        | 3              | 26             |
| 10   | 54         | 0.5           | 0.7        | 2              | 10             |
| 20-a | 95         | 0.5           | 0.8        | 3              | 16             |

Table 7 shows the result of the statistical test on APFD for all the objects. Except for *ant*, the results are not statistically significant. It could be concluded that the specified values for *stp* parameter do not have a significant impact on the APFD metric. The primary impact of the chosen *stp* values is on the running-times, as shown is Table 8. As the *stp* parameter increases, the running-times decrease almost linearly (*adjustive* parameters are on the average about one half of their maximum values). Considering that APFD values are similar for different *stp* values, using larger numbers for this parameter can improve the cost effectiveness. However, it should be kept in mind that the chosen *stp* values in this study are relatively low. If we choose very large values, the BNA technique will almost loose its feedback characteristic. Even with the current *stp* parameters, it is evident in Figure 3 that lower values result in better APFD

(although not statistically significant). Naturally, if we keep increasing the *stp*, at some point the APFD will become significantly lower.

## 6 Discussions on the Results

The reported results can provide useful lessons for both researchers and practitioners. They provide insight into the research questions presented in Section 4.

Regarding the first research question, impacts of using feedback mechanism, the obtained results present an interesting pattern. While using feedback results in earlier fault detection for TSL-based test suits, for Junit test suites, it does not affect the performance significantly. This could be attributed to different natures of test cases among these two families. Junit test developers are encouraged to target one and only one functionality of the system in each test case, and hence, different test cases are less likely to cover similar parts of the code. In TSL, for one specific operation multiple test cases with different parameters could be generated, and hence, a larger amount of redundancy can be potentially present. Therefore, the use of feedback for JUnit test suits does not improve the performance as much as it does for TSL (the feedback mechanism is of use only when there are test cases which cover same blocks of code). The feedback mechanism, however, increases the (machine) running-time always, and consequently, if this factor matters, testers need to be more careful about the feedback mechanism.

Regarding the second research question, effects of different information gathering techniques, it appears that the alternative approaches are hardly any different. Especially, in the case of change information, the differences are never significant. This could be due to the fact that all those techniques extract similar information from the system. The fact that each of those sources of information builds only one proportion of the whole model also contributes to their low impact. Nevertheless, the fault-proneness models were occasionally influential, and hence, it can be beneficial to investigate which model fits a specific program in the early phases of a project. Furthermore, there is a handful of other fault-proneness models and metrics that we did not investigate in this work. An important point to keep in mind is that the faults used in this experiment are all mutation faults. The process by which mutants are created does not take into the account the quality of the code. Hence, the distribution of faults may be different from that of a real system, and as a result the quality metrics may loose their prediction capability for faulty code spots. This suggest future experiments should focus on real faults as opposed to mutation faults.

Regarding the third research question, the effects of different parameters, we observed that the running time can be considerably affected. In practice, it is crucial to find the inference algorithm that suits a particular system.

Based on the observed data and mentioned justifications, when dealing with large systems which have a large number of classes, exact algorithms are not recommended. We also observed that performing inference for each single test case when using feedback is not needed. Adding several test cases after each inference can accelerate the technique while affecting the fault detection rate only slightly.

Finally, the obtained results suggest that the impact of one particular variable can differ from one system to another; therefore, in practice, one can use a similar methodology to investigate the impact of different variables in their specific domains. The fact that the influential and non-influential variables are spotted in this study can help such investigations to focus on the important factors. The existence of many non-influential variables is a valuable finding of this study. Besides being of help for future experiments, it demonstrates relative stability of the approach.

## 7 Conclusions and Future Work

In this paper, we have presented the results from a comprehensive empirical study regarding the impacts of different variables which could potentially affect the performance of the BN approach to test case prioritization. We have also introduced some new features to the approach, most importantly a feedback loop. The results of the experiment suggest that feedback mechanism can lead to significant improvements. The other factors were found to have only occasionally significant impacts as far as early fault detection is concerned. However, they often affect the running-time considerably.

There are also ways to extend this research work. This study focused on the parameters of the BN technique but experiments comparing other approaches of prioritization can be of great practical importance. Also, the cost-benefit analysis can be done using more sophisticated models. Conducting more experiments using real faults, industrial subject programs, and other testing mechanisms are all desirable but costly experiments that can provide further insight regarding the prioritization techniques.

**Acknowledgment** - The authors would like to gratefully acknowledge Dr. Hyunsook Do for providing the data sets.

## References

- [1] L. Briand and J. Wüst. Empirical studies of quality models in object-oriented systems. *Advances in Computers*, 56(0):98–167, 2002.
- [2] S. R. Chidamber and C. F. Kemerer. Towards a metrics suite for object oriented design. In *OOPSLA*, pages 197–211, 1991.
- [3] C. S. Collberg, G. Myles, and A. Huntwork. Sandmark—a tool for software protection research. *IEEE Security & Privacy*, 1(4):40–49, 2003.
- [4] H. Do, S. Elbaum, and G. Rothermel. Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact. *Empirical Software Engineering: An International Journal*, 10(4):405–435, 2005.
- [5] H. Do and G. Rothermel. An empirical study of regression testing techniques incorporating context and lifetime factors and improved cost-benefit models. In *FSE*, pages 141–151, 2006.
- [6] H. Do and G. Rothermel. On the use of mutation faults in empirical assessments of test case prioritization techniques. *IEEE Trans. Softw. Eng.*, 32(9):733–752, 2006.
- [7] S. Elbaum, A. G. Malishevsky, and G. Rothermel. Test case prioritization: A family of empirical studies. *IEEE Trans. on Softw. Eng.*, 28(2):159–182, 2002.
- [8] T. Gyimothy, R. Ferenc, and I. Siket. Empirical validation of object-oriented metrics on open source software for fault prediction. *IEEE Transactions on Software Engineering*, 31(10):897–910, 2005.
- [9] J.-M. Kim and A. Porter. A history-based test prioritization technique for regression testing in resource constrained environments. In *ICSE*, pages 119–129, 2002.
- [10] S. L. Lauritzen and D. J. Spiegelhalter. Local computations with probabilities on graphical structures and their application to expert systems. *J. Royal Stat. Soc., Series B (Methodological)*, 50(2):157–224, 1988.
- [11] D. Leon and A. Podgurski. A comparison of coverage-based and distribution-based techniques for filtering and prioritizing test cases. In *ISSRE*, pages 442–453, 2003.
- [12] S. Mirarab and L. Tahvildari. A prioritization approach for software test cases on Bayesian networks. In *FASE, LNCS 4422-0276*, pages 276–290, 2007.
- [13] T. Ostrand and M. J. Balcer. The category-partition method for specifying and generating functional tests. *Comm. ACM*, 31(6):676–688, 1988.
- [14] J. Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1988.
- [15] F. L. Ramsey and D. W. Schafer. *The Statistical Sleuth*. Duxbury Press, 1st edition, 1997.
- [16] G. Rothermel and M. J. Harrold. A safe, efficient regression test selection technique. *ACM Trans. on Softw. Eng. and Methodology*, 6(2):173–210, 1997.
- [17] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold. Prioritizing test cases for regression testing. *IEEE Trans. on Softw. Eng.*, 27(10):929–948, 2001.
- [18] D. Saff and M. D. Ernst. Reducing wasted development time via continuous testing. In *ISSRE*, pages 281–292, 2003.
- [19] A. Srivastava and J. Thiagarajan. Effectively prioritizing tests in development environment. In *ISSTA*, pages 97–106, 2002.
- [20] K. R. Walcott, M. L. Soffa, G. M. Kapfhammer, and R. S. Roos. Timeaware test suite prioritization. In *ISSTA*, pages 1–12, 2006.
- [21] W. E. Wong, J. R. Horgan, S. London, and H. A. Bellcore. A study of effective regression testing in practice. In *ISSRE*, pages 264–274, 1997.
- [22] C. Yuan and M. Druzdzel. An importance sampling algorithm based on evidence pre-propagation. In *Conf. Uncertainty in Art. Intel.*, pages 624–637, 2003.